S1 FUNCTIONAL REQUIREMENTS ENGINEERING

Detailed Engine Decomposition & Requirements Workbook

Common Team Information

Field	Student Entry
Team No.	10
Project Theme / Title	Centralized Workflow & Procurement Management System
Guide Name	Mrs. Indira Bidari
Date Prepared	04/10/2026

Sl. No.	SRN	Student Name
1	01FE24BCS103	Satish S Batakurki
2	01FE24BCS102	Swayambu V B
3	01FE24BCS135	Rakshita Kadam
4	01FE24BCS091	Aditya S Hegde
5		

SECTION A — DETAILED ENGINE DECOMPOSITION

1. What Students Must Establish Before Writing Requirements

Start from the approved S0 problem, stakeholders, pain points, scope and expected system behaviour. Identify the major technical responsibilities required to make the proposed system work. Group closely related responsibilities into engines that can be independently explained, owned, implemented and tested.

At this point, the engine list is a justified technical decomposition of the approved problem. After the SRS is prepared, students must return to the engine section and add the final requirement IDs supported by each engine. This keeps the decomposition before requirements while preserving traceability.

2. Engine Decomposition Rule

Step	Student Action	Evidence to Write
1. Problem Function	Identify a major system function or technical challenge from S0.	State the function/problem clearly.
2. Responsibility Group	Group functions that need the same technical mechanism, state or control.	Explain why the functions belong together.
3. Technical Responsibility	Convert the group into one precise engineering responsibility.	Write one responsibility statement beginning with “This engine is responsible for…”
4. Engine Boundary	Decide what the engine owns and what it must not own.	List in-scope and out-of-scope responsibilities.
5. Engine Definition	Name the engine based on the technical responsibility, not a screen/layer.	Give engine name, purpose, I/O, state, mechanism, failures and KPI.
6. Ownership	Assign one primary student owner.	Owner must defend the engine end-to-end.
7. Requirement Link	After SRS completion, map FR/NFR IDs to the engine.	Update the requirement mapping fields before Gate 1.

4. Proposed Engine Summary — Before Requirements

Engine ID	Proposed Engine Name	Major course Covered	Technical Problem Owned	Primary Responsibility	Primary Student Owner
E1	Procurement Workflow Engine	Web Tech + Software Engineering.	Requests skip steps or are handled by the wrong person.	Login, roles, request status flow and notifications.	Satish S Batakurki (01FE24BCS103)
E2	Vendor Selection & Decision Engine	DSA / DAA	Finding one vendor who can supply the full quantity.	Search, filter and select the vendor; track vendor replies.	Swayambu V B (01FE24BCS102)
E3	Data ,Transaction & Integrity Engine	DBMS	Records get half-saved, mismatched or changed without a trace.	Safe saving, order-receipt-invoice check, audit trail, reports.	Rakshita Kadam (01FE24BCS135)
E4	Concurrent Stock & Resource Engine	OS + DBMS	Two users can change the same stock at the same time.	Re-check stock before ordering; keep stock correct.	Aditya S Hegde (01FE24BCS091)

EVIDENCE 1 OF 7 — SOFTWARE REQUIREMENTS SPECIFICATION (SRS)
Stage S1 — Functional Requirements Engineering | General, Theme-Independent

1. Functional Requirements (FR)

List every functional requirement. Add rows as needed.

FR ID	Description	Priority (Mandatory/Challenge)	Source Need (from S0)
FR-01	Manage the full procurement flow: Pending → Approved → Vendor Selected → Ordered → Received → Completed. No skipped steps.	Mandatory	Need one clear procurement process
FR-02	Search vendors by product, quantity, availability and price; hide vendors who cannot supply the full amount.	Mandatory	Need to find suitable vendors fast
FR-03	Select one vendor who can supply the full quantity. Never split a request across vendors.	Mandatory	Need simple single-vendor orders
FR-04	Re-check vendor stock just before confirming the order.	Mandatory	Need to avoid orders on old stock data
FR-05	Keep stock correct when many users update it at the same time.	Mandatory	Need correct inventory
FR-06	Save related operations together; if one fails, undo all (commit / rollback).	Mandatory	Need consistent records
FR-07	Match purchase order, received quantity and invoice; allow payment only after they match.	Mandatory	Need to avoid wrong payments
FR-08	Give each role (Customer, Shopkeeper, Vendor, Warehouse, Finance) only its allowed actions.	Mandatory	Need to protect sensitive actions
FR-09	Record who did what and when for every important action. Records cannot be changed.	Mandatory	Need full traceability
FR-10	Show the current status of each request and where it is waiting or delayed.	Mandatory	Need visibility of progress
FR-11	Reports on pending requests, costs, vendor activity and delays.	Challenge	Need insight for management
FR-12	Reject invalid quantities and steps; no duplicate or orphan records.	Mandatory	Need reliable data
FR-13	Login / logout; only valid users can enter.	Mandatory	Need secure access
FR-14	Notify users of key events (approved, ordered, received, paid) and alert on problems.	Challenge	Need timely updates
FR-15	Send enquiries to vendors and track their replies (confirm / reject / availability).	Challenge	Need organised vendor contact

2. Non-Functional Requirements (NFR)

List performance, security, usability, reliability, and other quality requirements.

NFR ID	Description	Category	Measurable Target
NFR-01	Stock stays correct with many users at once.	Reliability	0 wrong stock counts in a 50-user test
NFR-02	A failed operation leaves no half-saved data.	Reliability	100% rolled back on failure
NFR-03	Only logged-in users with the right role can act.	Security	100% unauthorised attempts blocked
NFR-04	Vendor search is fast.	Performance	Within 2 seconds
NFR-05	Audit records cannot be edited or deleted.	Security	0 successful changes to audit records
NFR-06	Easy for a new user to create a request.	Usability	Done in under 5 minutes
NFR-07	The system should have a modular design so that each module can be changed or tested separately.	Maintainability	100% of major modules can be modified/tested independently
NFR-08		Auditability	100% of important actions are logged and 0 unauthorized changes to audit records

EVIDENCE 2 OF 7 — USE CASES / USER STORIES
Stage S1 — Functional Requirements Engineering | General, Theme-Independent

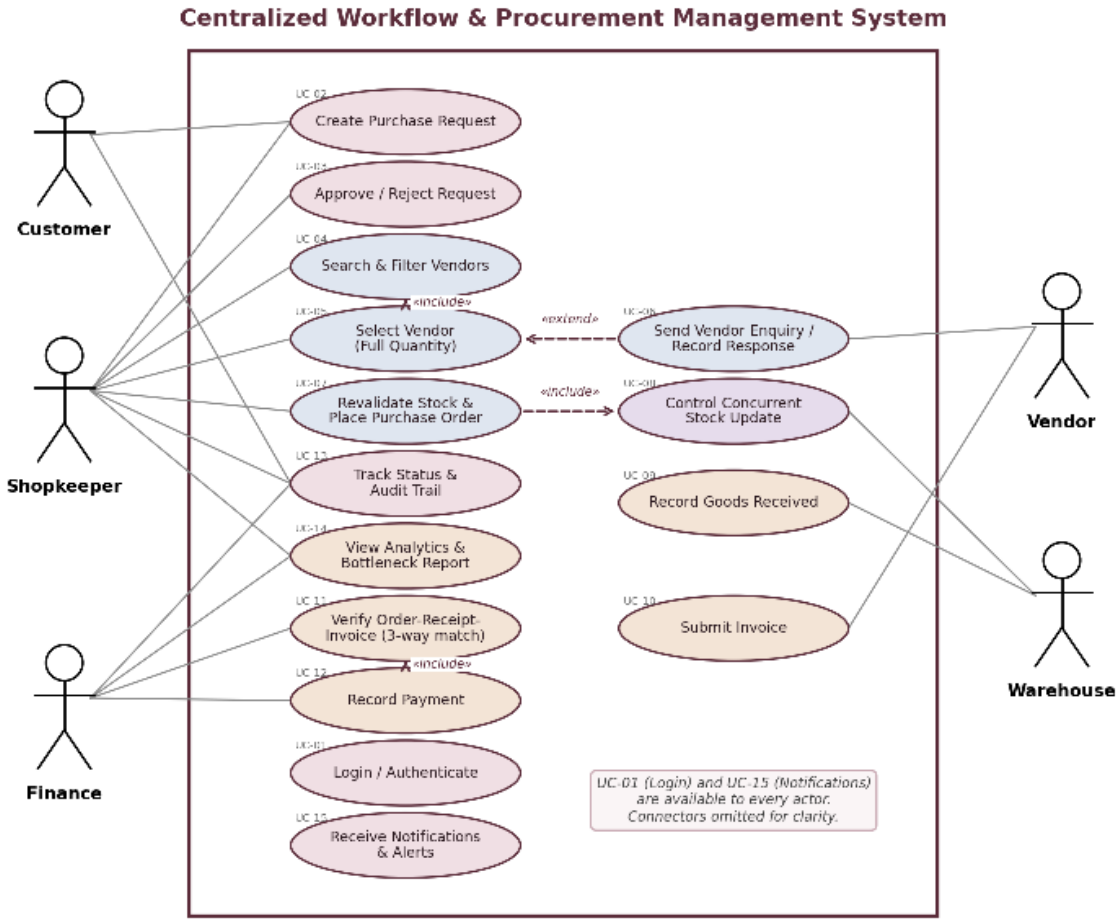
1. Use Case / User Story Table

One row per use case or user story. Cover normal, boundary, and exception flows.

ID	Actor	Description / User Story	Main Flow	Exception Flow
UC-01	All users	Log in to use the system.	Enter credentials → system verifies → role dashboard opens.	Wrong password: access denied.
UC-02	Customer / Shopkeeper	Create a purchase request.	Enter product and quantity → request saved as Pending with a unique ID.	Quantity is zero or negative: rejected.
UC-03	Shopkeeper	Approve or reject a request.	Open pending request → approve or reject → status updated.	Request not Pending: action blocked.
UC-04	Shopkeeper	Search and filter vendors.	Choose request → system lists vendors with full quantity.	No vendor has enough stock: show "no vendor".
UC-05	Shopkeeper	Select one vendor.	Compare price and availability → select vendor → status Vendor Selected.	Trying to split quantity across vendors: refused.
UC-06	Shopkeeper, Vendor	Send enquiry and record vendor reply.	Send enquiry → vendor replies → reply saved.	No reply: marked "No response".
UC-07	Shopkeeper	Place the purchase order.	System re-checks stock → order created → status Ordered.	Stock changed and too low: order blocked.
UC-08	System	Handle users updating stock together.	System lets one update finish before the next → stock stays correct.	Two users want the last item: first gets it, second is told.
UC-09	Warehouse	Record goods received.	Select order → enter quantity received → status Received.	Quantity differs from order: flagged.
UC-10	Vendor	Submit an invoice.	Select order → enter invoice details → linked to order.	Duplicate invoice: rejected.
UC-11	Finance	Check order, receipt and invoice.	System compares the three → marks as matched.	Mismatch: payment blocked.
UC-12	Finance	Record payment.	Select matched invoice → record payment → status Completed.	Not yet matched: payment refused.
UC-13	Customer, Shopkeeper, Finance	Track request status and history.	Enter request ID → see status and history.	No permission: access denied.
UC-14	Shopkeeper, Finance	View reports.	Choose report → see pending, cost, vendor activity, delays.	No data: empty report shown.
UC-15	All users	Receive notifications.	Event happens → relevant users notified.	Delivery fails: retried.

2. Use Case Diagram

.


Centralized Workflow & Procurement Management System
Customer
Shopkeeper
Finance
Vendor
Warehouse
UC 02
Create Purchase Request
UC-03
Approve / Reject Request
UC 04
Search & Filter Vendors
UC-05
Select Vendor (Full Quantity)
UC-06
Send Vendor Enquiry / Record Response
UC-07
Revalidate Stock & Place Purchase Order
UC-08
Control Concurrent Stock Update
UC 12
Track Status & Audit Trail
UC 09
Record Goods Received
UC-14
View Analytics & Bottleneck Report
UC 11
Verify Order-Receipt-Invoice (3-way match)
UC 10
Submit Invoice
UC 13
Record Payment
UC-01
Login / Authenticate
UC 15
Receive Notifications & Alerts
«include»
«extend»
«include»
«include»
UC-01 (Login) and UC-15 (Notifications) are available to every actor. Connectors omitted for clarity.

EVIDENCE 3 OF 7 — ACCEPTANCE CRITERIA SHEET
Stage S1 — Functional Requirements Engineering | General, Theme-Independent

1. Acceptance Criteria per Requirement

For every requirement, state the specific, testable condition(s) that define it as 'done'.

Requirement ID	Acceptance Criteria	Test / Verification Method
FR-01	Steps happen only in the correct order; skipped or backward steps are rejected; each request has a unique ID.	Test every valid and invalid step change.
FR-02	Search returns matching vendors; vendors without full quantity are excluded.	Search with sample vendor data.
FR-03	A vendor qualifies only if stock ≥ requested quantity; one vendor per request.	Test stock = qty−1, qty, qty+1.
FR-04	If stock changed before confirmation, the order is blocked and user is alerted.	Change stock, then try to order.
FR-05	Simultaneous updates never give wrong quantities or lost updates.	Run many orders at once; check final stock.
FR-06	Failed operations leave no half-saved data.	Force a failure mid-operation; check nothing was saved.
FR-07	Order, receipt and invoice are linked; mismatches detected; payment only after match.	Test matching and mismatching records.
FR-08	Users can do only what their role allows.	Try each action with each role.
FR-09	Every important action is logged with who, what, when; logs cannot be edited.	Perform actions and check the log.
FR-10	Current status is always shown; delayed requests are visible.	Move a request through steps and check status.
FR-11	Reports show correct pending count, cost, vendor activity and delays.	Compare report with known data.
FR-12	Invalid, duplicate and orphan records are rejected.	Try to insert bad data.
FR-13	Only valid users can log in; logout ends the session.	Try valid and invalid logins.
FR-14	Right users are notified for each key event.	Trigger each event.
FR-15	Enquiries and replies are saved with their status.	Send enquiry; test each reply type.
NFR-01	No wrong stock counts in a 50-user test.	Automated concurrent test.
NFR-02	100% rollback on failure.	Force failures and check.
NFR-03	All unauthorised attempts blocked.	Try access without login / wrong role.
NFR-04	Vendor search finishes within 2 seconds.	Time the searches.
NFR-05	No audit record can be changed or deleted.	Try to edit/delete a log entry.
NFR-06	New user creates a request in under 5 minutes.	Test with first-time users.

EVIDENCE 4 OF 7 — REQUIREMENT PRIORITY TABLE
Stage S1 — Functional Requirements Engineering | General, Theme-Independent

1. Priority Classification

Requirement ID	Description	Priority	Justification
FR-01	Procurement workflow	Mandatory	Core requirement; needed for acceptance.
FR-02	Vendor search & filtering	Mandatory	Core requirement; needed for acceptance.
FR-03	Single-vendor full-quantity selection	Mandatory	Core requirement; needed for acceptance.
FR-04	Stock re-check before order	Mandatory	Core requirement; needed for acceptance.
FR-05	Concurrent stock update control	Mandatory	Core requirement; needed for acceptance.
FR-06	Atomic transactions	Mandatory	Core requirement; needed for acceptance.
FR-07	Order–receipt–invoice matching	Mandatory	Core requirement; needed for acceptance.
FR-08	Role-based access	Mandatory	Core requirement; needed for acceptance.
FR-09	Audit trail	Mandatory	Core requirement; needed for acceptance.
FR-10	Status tracking	Mandatory	Core requirement; needed for acceptance.
FR-11	Reports & delay detection	Challenge	Extra feature; system works without it.
FR-12	Data integrity	Mandatory	Core requirement; needed for acceptance.
FR-13	User login	Mandatory	Core requirement; needed for acceptance.
FR-14	Notifications & alerts	Challenge	Extra feature; system works without it.
FR-15	Vendor coordination	Challenge	Extra feature; system works without it.
NFR-01	Stock stays correct with many users at once	Mandatory	Core requirement; needed for acceptance.
NFR-02	A failed operation leaves no half-saved data	Mandatory	Core requirement; needed for acceptance.
NFR-03	Only logged-in users with the right role can act	Mandatory	Core requirement; needed for acceptance.
NFR-04	Vendor search is fast	Mandatory	Core requirement; needed for acceptance.
NFR-05	Audit records cannot be edited or deleted	Mandatory	Core requirement; needed for acceptance.
NFR-06	Easy for a new user to create a request	Challenge	Extra feature; system works without it.

EVIDENCE 5 OF 7 — REQUIREMENTS TRACEABILITY MATRIX (RTM)
Stage S1 — Functional Requirements Engineering | General, Theme-Independent

1. Traceability Chain

Trace every requirement from its source need through to design, implementation and test.

Req ID	Source Need (S0)	Owning Engine/Module	Design Ref (S2–S4)	Test Ref (S6)	Status
FR-01	Need one clear procurement process	E1 – Workflow	TBD	TBD	Specified
FR-02	Need to find suitable vendors fast	E2 – Vendor Selection	TBD	TBD	Specified
FR-03	Need simple single-vendor orders	E2 – Vendor Selection	TBD	TBD	Specified
FR-04	Need to avoid orders on old stock data	E4 – Stock Control	TBD	TBD	Specified
FR-05	Need correct inventory	E4 – Stock Control	TBD	TBD	Specified
FR-06	Need consistent records	E3 – Data & Transaction	TBD	TBD	Specified
FR-07	Need to avoid wrong payments	E3 – Data & Transaction	TBD	TBD	Specified
FR-08	Need to protect sensitive actions	E1 – Workflow	TBD	TBD	Specified
FR-09	Need full traceability	E3 – Data & Transaction	TBD	TBD	Specified
FR-10	Need visibility of progress	E1 – Workflow	TBD	TBD	Specified
FR-11	Need insight for management	E3 – Data & Transaction	TBD	TBD	Specified
FR-12	Need reliable data	E3 – Data & Transaction	TBD	TBD	Specified
FR-13	Need secure access	E1 – Workflow	TBD	TBD	Specified
FR-14	Need timely updates	E1 – Workflow	TBD	TBD	Specified
FR-15	Need organised vendor contact	E2 – Vendor Selection	TBD	TBD	Specified
NFR-01	Stock stays correct with many users at once	E4 – Stock Control	TBD	TBD	Specified
NFR-02	A failed operation leaves no half-saved data	E3 – Data & Transaction	TBD	TBD	Specified
NFR-03	Only logged-in users with the right role can act	E1 – Workflow	TBD	TBD	Specified
NFR-04	Vendor search is fast	E2 – Vendor Selection	TBD	TBD	Specified
NFR-05	Audit records cannot be edited or deleted	E3 – Data & Transaction	TBD	TBD	Specified
NFR-06	Easy for a new user to create a request	E1 – Workflow	TBD	TBD	Specified

REMINDER: This is a LIVE document — update it at every subsequent gate as design and test references become available.

ENGINE DECOMPOSITION TEMPLATE — STUDENT ENTRY
Stage S1 — Functional Requirements Engineering | Gate 1 Requirements Baseline

Purpose: derive independently ownable and testable technical engines from approved requirements before architecture or coding begins.

1. Requirement Clustering Sheet

Group related requirements by common technical responsibility. Do not cluster only by UI screen, frontend/backend layer, or convenience of work division.

Cluster ID	Cluster / Responsibility Area	Requirement IDs	Common Technical Responsibility	Engine
C1	Workflow & User Interaction	FR-01, 08, 10, 13, 14	Users, roles, status flow and notifications	E1 – Procurement Workflow Engine
C2	Vendor Search & Selection	FR-02, 03, 15	Search, filter and pick one vendor	E2 – Vendor Selection & Decision Engine
C3	Data, Transactions & Integrity	FR-06, 07, 09, 11, 12	Safe saving, matching, audit and reports	E3 – Data, Transaction & Integrity Engine
C4	Concurrent Stock Control	FR-04, 05	Keep stock correct with many users	E4 – Concurrent Stock & Resource Engine
REMINDER: Use the decomposition flow: Requirement → Cluster → Technical Responsibility → Engine → Student Owner.				

2. Engine Decomposition Matrix

Define each engine as a technical unit with a clear responsibility, inputs/outputs, owned state, mechanism, failure behaviour, measurable KPI and one primary student owner.

Engine ID	Engine Name	Requirement Cluster(s)	Primary Technical Responsibility	Inputs	Outputs	State / Data Owned	Core Mechanism / Algorithm	Dependencies / Interface	Failure Cases / Recovery	Primary KPI
E1	Procurement Workflow Engine	C1	Login, roles, status flow, notifications	Login, requests, approvals	Status changes, notifications	Users, roles, request status	Allowed-steps table; role check	Uses E3 to save; gets results from E2, E4	Invalid step or role: rejected	100% invalid steps blocked
E2	Vendor Selection & Decision Engine	C2	Search, filter and select one vendor	Product, quantity, price	Eligible vendors, chosen vendor	Vendor lists, enquiry replies	Keep vendors with enough stock; sort by price	Asks E4 to re-check stock	No vendor: show message	Search within 2 s
E3	Data Transaction & Integrity Engine	C3	Safe saving, matching, audit trail, reports	Orders, receipts, invoices, payments	Saved records, mismatch reports, logs	Orders, invoices, payments, audit log	Commit / rollback; 3-way match; constraints	Used by E1, E2, E4	Failure: undo everything	0 half-saved records
E4	Concurrent Stock & Resource Engine	C4	Re-check stock; stop clashing updates	Stock check and update requests	Stock OK / changed; updated stock	Stock quantities, locks	Locking so one update at a time	Called by E2 and E1 before ordering	Two users, last item: first wins	0 wrong stock counts
REMINDER: A valid engine must be independently explainable, own a distinct technical responsibility, and be verifiable through measurable evidence.										

ENGINE 01 — DESCRIPTION SHEET S1 Engine Definition Template | Complete only for engines used by the team	
Engine ID	E1
Engine Name	Procurement Workflow Engine
Primary Student Owner — SRN	01FE24BCS135
Primary Student Owner — Name	Rakshita Kadam
Requirement Cluster(s) / Requirement IDs	C1 – FR-01, 08, 10, 13, 14

A. Engine Purpose and Description

In 3–5 sentences, state what this engine is responsible for, why it exists, and what technical problem it solves. Keep the description at S1 level; do not present detailed architecture or code.

This engine handles login, user roles and the request status flow. It makes sure every request follows the correct steps from Pending to Completed. It also tracks status and sends notifications.

B. Technical Responsibility and Boundary

Primary Responsibility	What This Engine Owns	What This Engine Does NOT Own
Control who can do what, and move requests through valid steps.	• Users and roles • Status flow and rules • Status tracking • Notifications	• Vendor selection (E2) • Stock control (E4) • Saving data and audit (E3)

C. Inputs, Outputs and State

Inputs / Trigger	Outputs / Result	State / Data Owned
Login, new request, approve / reject	Status changes, notifications, allow / deny	Users, roles, request status
Events from other engines	Alerts to users	Notification list

D. Core Technical Mechanism

Algorithm / Mechanism / Rule	Why It Is Needed	Requirement(s) Supported
Allowed-steps table	Stops skipped or backward steps	FR-01, FR-10
Role check on every action	Blocks unauthorised actions	FR-08, FR-13

E. Dependencies, Failure Behaviour and KPI

Interface / Dependency	Possible Failure / Boundary Case	Expected Handling / Recovery	KPI / Measurement
E2, E3, E4	Invalid step attempted	Reject and keep old status	100% invalid steps blocked
Role list	Wrong role tries an action	Deny and log	0 unauthorised actions

REMINDER: At Gate 1, explain the engine responsibility, ownership and measurable success criteria. Detailed architecture, API implementation, database tables and code belong to later stages.

ENGINE 02 — DESCRIPTION SHEET S1 Engine Definition Template | Complete only for engines used by the team	
Engine ID	E2
Engine Name	Vendor Selection & Decision Engine
Primary Student Owner — SRN	01FE24BCS103
Primary Student Owner — Name	Satish S B
Requirement Cluster(s) / Requirement IDs	C2 – FR-02, 03, 15

A. Engine Purpose and Description

In 3–5 sentences, state what this engine is responsible for, why it exists, and what technical problem it solves. Keep the description at S1 level; do not present detailed architecture or code.

This engine finds vendors and picks one vendor who can supply the full quantity. It removes vendors with too little stock and compares the rest by price. It also tracks enquiries and replies from vendors.

B. Technical Responsibility and Boundary

Primary Responsibility	What This Engine Owns	What This Engine Does NOT Own
Search, filter and select one vendor.	• Vendor search • Full-quantity check • Vendor selection • Enquiry replies	• Changing stock (E4) • Saving data (E3) • Login and status (E1)

C. Inputs, Outputs and State

Inputs / Trigger	Outputs / Result	State / Data Owned
Product, quantity, price	Eligible vendors, chosen vendor	Vendor lists
Enquiry and vendor reply	Reply status	Enquiry records

D. Core Technical Mechanism

Algorithm / Mechanism / Rule	Why It Is Needed	Requirement(s) Supported
Keep only vendors with stock ≥ quantity	Ensures full-quantity supply	FR-02, FR-03
Sort remaining vendors by price	Picks the best vendor	FR-03

E. Dependencies, Failure Behaviour and KPI

Interface / Dependency	Possible Failure / Boundary Case	Expected Handling / Recovery	KPI / Measurement
E3, E4, E1	No vendor has enough stock	Show "no vendor"	Unsuitable vendors always excluded
Vendor reply	Vendor does not reply	Mark "No response"	All enquiries have a status

REMINDER: At Gate 1, explain the engine responsibility, ownership and measurable success criteria. Detailed architecture, API implementation, database tables and code belong to later stages.

ENGINE 03 — DESCRIPTION SHEET S1 Engine Definition Template | Complete only for engines used by the team	
Engine ID	E3
Engine Name	Data,Transaction & Integrity Engine
Primary Student Owner — SRN	01FE24BCS091
Primary Student Owner — Name	Aditya S Hegde
Requirement Cluster(s) / Requirement IDs	C3 – FR-06, 07, 09, 11, 12

A. Engine Purpose and Description

In 3–5 sentences, state what this engine is responsible for, why it exists, and what technical problem it solves. Keep the description at S1 level; do not present detailed architecture or code.

This engine saves procurement data safely. It makes sure operations are saved fully or not at all, checks that order, receipt and invoice match, and keeps a permanent record of every action. It also produces reports.

B. Technical Responsibility and Boundary

Primary Responsibility	What This Engine Owns	What This Engine Does NOT Own
Keep procurement data correct and traceable.	• Orders, receipts, invoices, payments • Audit log • Data rules • Reports	• Status rules and roles (E1) • Vendor selection (E2) • Stock locking (E4)

C. Inputs, Outputs and State

Inputs / Trigger	Outputs / Result	State / Data Owned
Order, receipt, invoice, payment data	Saved records, mismatch report	Orders, invoices, payments
Important actions from other engines	Audit log entries	Audit log

D. Core Technical Mechanism

Algorithm / Mechanism / Rule	Why It Is Needed	Requirement(s) Supported
Commit / rollback	No half-saved data	FR-06
Order-receipt-invoice match	Stops wrong payments	FR-07

E. Dependencies, Failure Behaviour and KPI

Interface / Dependency	Possible Failure / Boundary Case	Expected Handling / Recovery	KPI / Measurement
E1, E2, E4	Failure while saving	Undo everything	0 half-saved records
Order, receipt, invoice	Quantity or amount mismatch	Block payment	100% mismatches found

REMINDER: At Gate 1, explain the engine responsibility, ownership and measurable success criteria. Detailed architecture, API implementation, database tables and code belong to later stages.

ENGINE 04 — DESCRIPTION SHEET S1 Engine Definition Template | Complete only for engines used by the team	
Engine ID	E4
Engine Name	Concurrent Stock and Resource Engine
Primary Student Owner — SRN	01FE24BCS102
Primary Student Owner — Name	Swyambu V B
Requirement Cluster(s) / Requirement IDs	C4 – FR-04, 05

A. Engine Purpose and Description

In 3–5 sentences, state what this engine is responsible for, why it exists, and what technical problem it solves. Keep the description at S1 level; do not present detailed architecture or code.

This engine keeps stock correct when many users work together. It re-checks stock just before an order is confirmed and lets only one stock update happen at a time. If something fails, it restores the earlier stock.

B. Technical Responsibility and Boundary

Primary Responsibility	What This Engine Owns	What This Engine Does NOT Own
Re-check stock and stop clashing stock updates.	• Stock quantities • Locks on stock • Stock re-check • Undo on failure	• Vendor choice (E2) • Status and roles (E1) • Audit and reports (E3)

C. Inputs, Outputs and State

Inputs / Trigger	Outputs / Result	State / Data Owned
Stock check before order	Stock OK / changed	Stock quantities
Many users updating same item	One update at a time; correct stock	Locks

D. Core Technical Mechanism

Algorithm / Mechanism / Rule	Why It Is Needed	Requirement(s) Supported
Locking so one update at a time	Stops lost updates	FR-05
Re-check stock before confirming	Avoids orders on old data	FR-04

E. Dependencies, Failure Behaviour and KPI

Interface / Dependency	Possible Failure / Boundary Case	Expected Handling / Recovery	KPI / Measurement
E2, E1, E3	Two users want last item	First succeeds, second told	0 wrong stock counts
Stock records	Stock changed after selection	Block order and alert	All changes detected

REMINDER: At Gate 1, explain the engine responsibility, ownership and measurable success criteria. Detailed architecture, API implementation, database tables and code belong to later stages.